CONFIDENTIAL · ENGINEERING PACK

Alehu MVP — Technical Documentation

Next.js frontend, Flask API, PostgreSQL, wearable ingest, and hardware bill of materials.

Architecture

- **Frontend:** Next.js App Router, TypeScript, Tailwind. Marketing pages (including /hardware) are statically renderable with SEO, generated Open Graph PNG, sitemap, and robots rules. Guardian dashboard is a client app against the API.
- **Backend:** Python 3.12, Flask 3, SQLAlchemy 2, Flask-Migrate, JWT, Marshmallow, CORS.
- **Database:** PostgreSQL. Core tables: users, roles, permissions, customers, devices, alerts.
- **Wearable path:** firmware POSTs compact telemetry to `/api/ingest/telemetry` using JWT or `X-Device-Key`.

API surface

Area	Endpoints
Health / contact	GET /api/health · POST /api/contact
Auth	register, login, refresh, logout, me
Operations	users, customers, devices, roles, permissions
Safety	alerts CRUD + acknowledge · dashboard/summary
Ingest	POST /api/ingest/telemetry

Device telemetry fields

devices now carry firmware_version, battery_level, last_latitude, last_longitude, and last_seen. Optional alert_type on ingest creates a structured safety event (geofence, fall, sos, battery, distress).

Security notes

- Passwords hashed with bcrypt. JWT access + refresh. Logout blocklists JTI (replace with Redis in production).
- CORS locked to the Next.js origin via CORS_ORIGINS.
- No raw audio or image payload is accepted by the MVP API.
- Rotate SECRET_KEY, JWT_SECRET_KEY, DEVICE_INGEST_KEY, and database passwords before any public deploy.

Local run

Backend: create venv, pip install -r backend/requirements.txt, copy .env, create database alehu, flask db upgrade, python run.py --seed, python run.py.

Frontend: cd frontend, npm install, copy .env.example to .env.local, npm run dev. Open http://localhost:3000 and sign in with admin@alehu.com / Admin123!

Hardware guidance — Alehu Band v1

Design goal: a child-safe, water-resistant wearable that can classify distress and motion on-device, then send a tiny encrypted event. Prefer parts that are globally sourcable and already have Ethiopia-friendly LTE-M/NB-IoT options.

Function	Recommended starting parts
Main MCU	ESP32-S3 (Wi-Fi/BLE prototyping) or nRF52840 (BLE + better battery)
Cellular (field)	SIM7080G or Quectel BG95 LTE-M/NB-IoT + nano-SIM
GNSS	Quectel L76-L or ATGM336H GNSS module with ceramic patch antenna
IMU (fall/impact)	Bosch BMI270 or InvenSense ICM-42670-P (low-power 6-axis)
Microphone	Knowles analog MEMS + local keyword/scream model. Do not stream audio.
SOS button	Sealed tactile switch, recessed, double-press firmware debounce
Battery	400–800 mAh LiPo, protection PCB, USB-C charge via TP4056 or BQ series
Power path	3.3V LDO or buck; fuel-gauge optional (MAX17048)
Enclosure	IP67 TPU/PC blend, rounded, no removable small parts, skin-safe finish
Indicators	One status LED behind diffuser; optional haptic (coin ERM)

Firmware responsibilities

- Sample IMU at 50–100 Hz; run a tiny fall/impact classifier on MCU.
- Run a small acoustic model locally; transmit only event labels, never WAV/PCM.
- Maintain geofence polygons pushed from the cloud; alert on exit + dwell.
- Heartbeat every 30–120 seconds: serial, battery, lat/lng, firmware version.
- On SOS/fall/distress: high-priority ingest with severity=critical.
- Deep sleep between samples to target 3–5 day battery.

Prototype sequence

- Week 1–2: ESP32-S3 + BMI270 + USB power. Prove fall + button events into /api/ingest/telemetry.
- Week 3–4: add GNSS and geofence firmware. Show live last-known point in the dashboard.
- Week 5–6: battery + enclosure mock. Measure current in sleep vs alert burst.
- Week 7+: swap in LTE-M module for independent-from-phone coverage. Start school pilot with 10 units.

Safety and compliance (plan, not yet certified)

Target IP67, IEC 62115 / toy-adjacent mechanical safety, battery UN38.3 for shipping, RF (CE/FCC or local ETA as applicable), and a child-data policy that matches the privacy page: no raw media retention. Certify before paid school rollout.